

Unit 7: Security in Web Applications 8 hours

Compiled By

Lecturer, Sunil Bahadur Bist

NAST, Dhangadhi

Contents

- **Web application security fundamentals**
 - Foundations of security
 - Vulnerabilities
 - Threats
 - Attacks
- **Security Principles**
 - Least Privilege
 - Defense in Depth
 - Fail Securely

Contents

- **Common web vulnerabilities**

- SQL Injection
- XSS
- CSRF
- DoS
- Live examples and demonstrations of vulnerabilities

- **Security best practices for web development**

- Input Validation and Output Encoding
- Secure Authentication and Authorization
 - Multi-Factor Authentication (MFA), Role-Based Access Control (RBAC),

Contents

Secure Session Handling, Session Fixation and Hijacking Prevention,
Error Handling and Logging, Secure File Uploads

- SSL/TLS and HTTPS
- **Firewalls, proxies, and VPNs**
- **Host and Network Security threats and countermeasures**
- **Application Security threats and countermeasures**
- **Design guidelines for secure web applications**

Web application security fundamentals

- Web application security fundamentals are essential for protecting the safety of
 - Websites,
 - web applications,
 - web services and
 - Microservices
- against attacks, unauthorized access, data breaches, and other risks by focusing on core principles, vulnerabilities, threats, and preventive measures.

Foundations of security

- Foundations of security are the core principles that guides the protection of information systems and web applications from attacks.
- It ensure that the data and services remain secure, trustworthy, and available.
- **Key Principles**
- **Confidentiality:** It ensures that the information's are accessible only to authorized user. For e.g., Encrypting user passwords and sensitive data.

Foundations of security

- **Integrity:** It Guarantees that the data remains accurate and unaltered. For eg. Using hash functions to verify data consistency.
- **Availability:** Ensures that systems and data are accessible when needed. For e.g., Implementing backup servers and DDoS protection.
- **Authentication:** Verifies the identity of users or systems. For e.g., Login systems using usernames, passwords, or biometrics.

Foundations of security

- **Authorization:** Determines what authenticated users are allowed to do. For e.g., Admins can access control panels where as users cannot.
- **Non-repudiation:** Prevents parties from denying their actions or transactions. For e.g., Digital signatures and audit logs.

Vulnerabilities

- Vulnerabilities are weaknesses or flaws in a web application that can be exploited by attackers to gain unauthorized access, steal data, or disrupt operations.
- Identifying and fixing these is crucial for securing web applications.
- **Common Vulnerabilities**
- **SQL Injection (SQLi)**
 - Inserting malicious SQL code into a query to manipulate the database.
 - Example: `SELECT * FROM users WHERE username = 'admin' --`
 - Prevention: Use parameterized queries or ORM frameworks like SQLAlchemy.

Vulnerabilities

- **Cross-Site Scripting (XSS)**

- Injecting malicious JavaScript into a web page viewed by others.
- Example: `<script>alert('Hacked!')</script>` in a comment field.
- Prevention: Sanitize input and use output encoding (e.g., HTML escaping).

- **Cross-Site Request Forgery (CSRF)**

- Tricks a logged-in user into performing unwanted actions.
- Example: A hidden form auto-submitting a fund transfer.
- Prevention: Use CSRF tokens and confirm actions for sensitive operations.

Vulnerabilities

- **Insecure Direct Object Reference (IDOR)**

- Accessing data by manipulating URLs or IDs.
- Example: Changing /invoice/1001 to /invoice/1002 to view others' invoices.
- Prevention: Implement access control checks on server side.

- **Security Misconfiguration**

- Using default settings, exposing debug info, or leaving ports open.
- Example: Showing detailed server errors to users.
- Prevention: Regularly audit settings and disable unnecessary features.

Vulnerabilities

- **Broken Authentication**

- Weak or flawed authentication mechanisms.
- Example: Session IDs in URLs, predictable password reset tokens.
- Prevention: Use secure authentication frameworks and implement MFA.

- **Sensitive Data Exposure**

- Improper handling of personal or financial information.
- Example: Storing passwords in plain text.
- Prevention: Encrypt sensitive data at rest and in transit.

Threats

- Threats are potential dangers or harmful events that can exploit vulnerabilities in a web application, leading to unauthorized access, data theft, system damage, or service disruption.

Web Security Threats

- **Phishing:** Aims to collect sensitive data like usernames, passwords, credit card numbers, or bank details. For e.g., A fake login page that looks like a real website.

Threats

- **Malware:** Malicious software designed to damage, steal, or disrupt systems. For e.g., Spyware stealing user credentials.
- **Man-in-the-Middle (MitM):** Attackers intercept and possibly alter communication between users and servers. For e.g., Capturing login data sent over unsecured HTTP.

Threats

- **Social Engineering:** Manipulating people into giving up confidential information. For e.g., Pretending to be an IT support agent to get login access.
- **Insider Threats::** Harm caused by people within the organization (e.g., employees). For e.g., A staff member leaking customer data.

Attacks

- Attacks are deliberate actions carried out by malicious users (attackers or hackers) to exploit vulnerabilities in a web application, aiming to steal data, disrupt services, or gain unauthorized access.
- **Common Web Attacks**
- **Brute Force Attack**
 - Repeatedly trying different username-password combinations to gain access.

Attacks

- For e.g., Guessing a user's password using automated scripts.
- Use account lockout, CAPTCHAs, and strong password policies to prevent from it.
- **Denial of Service (DoS) / Distributed DoS (DDoS)**
 - Overwhelms a web server with traffic to make it unavailable to users.
 - Example, Botnets sending millions of requests to crash a website.
 - Use firewalls, load balancers, and rate-limiting from prevent from it.

Attacks

- **Zero-Day Attack**

- Exploiting a vulnerability that is unknown to the software vendor and has no fix.
- For e.g., Attacking a newly discovered flaw in a web framework.
- Regular updates, monitoring, and use of security tools to prevent from it.

Security Principles

- Security principles are foundational concepts designed to protect information systems and data from threats.
- Here's a list of key security principles with concise explanations:
 - Least Privilege
 - Defense in Depth
 - Fail Securely

Least Privilege

- Users, systems, and processes should be granted only the minimum access or permissions necessary to perform their functions.
- It uses to reduces the attack surface.
- It limits damage in case of compromise.
- Prevents unauthorized access to sensitive systems or data.
- For example, A user who needs to view financial reports shouldn't have access to edit or delete them.

Defense in Depth

- A layered security approach where multiple defensive mechanisms are implemented across different levels to protect data and systems.
- If one layer fails, others still provide protection.
- Increases resilience against sophisticated or multi-stage attacks.
- For e.g., A network protected by a firewall, intrusion detection system (IDS), multi-factor authentication (MFA), and data encryption.

Fail Securely

- Systems should default to a secure state in the event of a failure or error, rather than compromising security.
- It Prevents exposure of sensitive data during crashes or errors.
- It Ensures that failure does not open a backdoor.
- For example, If a login system fails to connect to the authentication server, it should deny access rather than allow it by default.

Common web vulnerabilities

- SQL Injection
- XSS
- CSRF
- DoS
- Live examples and demonstrations of vulnerabilities

SQL Injection

- **SQL injection occurs when:**
 - An attacker inputs malicious SQL code into a form field or URL.
 - That input gets executed directly as part of a SQL query on the server.
- **Login Bypass**
- `SELECT * FROM users WHERE username = '{username}' AND password = '{password}'`

SQL Injection

- Test the SQL Injection
- Enter this in the form:
 - Username: ' OR '1'='1
 - Password: ' OR '1'='1
- The query always returns true, bypassing authentication!
- You'll be logged in without knowing the password, because the resulting query becomes:
- `SELECT * FROM users WHERE username = ' OR '1'='1' AND password = ' OR '1'='1'`

SQL Injection

- **To Protect from bypassing authentication**

Replace this line:

```
query = f"SELECT * FROM users WHERE username = '{username}' AND  
password = '{password}'"  
c.execute(query)
```

With

```
query = "SELECT * FROM users WHERE username = ? AND password = ?"  
c.execute(query, (username, password))
```

SQL Injection

- **Dumping Data**

Assume a URL: `http://example.com/products.php?id=2`

query = `f"SELECT * FROM products WHERE id = {product_id}"`

Attacker Input:

`http://example.com/products.php?id=2 OR 1=1`

Resulting Query:

`SELECT * FROM products WHERE id = 2 OR 1=1;`

All records from the products table will be returned!

SQL Injection

`http://example.com/products.php?id=1; DROP TABLE products;`

- This can delete the table, if semicolon execution is allowed.

- **To prevent from Dumping data**

`query = "SELECT * FROM products WHERE id = ?"`

`cursor.execute(query, (product_id,))`

SQL Injection

- **Data Exfiltration with UNION**

Input for Attack:

`http://example.com/product.php?id=1 UNION SELECT username, password FROM users`

Resulting SQL Query:

`SELECT * FROM products WHERE id = 1`

`UNION`

`SELECT username, password FROM users;`

SQL Injection

- **Blind SQL Injection**

You don't see results directly, but the page responds differently depending on true/false statements.

URL Input

`http://example.com/product.php?id=5 AND 1=1`

Then

`http://example.com/product.php?id=5 AND 1=2`

SQL Injection

- **Error-Based SQL Injection**

This uses errors to extract info from the database.

Input:

`http://example.com/product.php?id=1' AND extractvalue(1, concat(0x7e,(SELECT database()),0x7e))--+`

What Happens:

Database name is exposed in an error message!

How to Prevent These Vulnerabilities

Prevention Method	Description
Prepared Statements	Use parameterized queries (cursor.execute("SELECT * FROM users WHERE id = ?", (id,)))
ORM Libraries	Use SQLAlchemy, Django ORM, etc., which abstract SQL safely
Input Validation	Whitelist input formats (e.g., allow only digits for id)
Escaping User Input	Escaping dangerous characters using library functions
Least Privilege	Database users should have minimal access (e.g., no DROP)
Web Application Firewalls (WAF)	Block common injection attempts

CSRF

DoS

Security best practices for web development

- Security best practices for web development refer to a set of guidelines, techniques, and strategies used to protect web applications from threats, vulnerabilities, and unauthorized access.
- **Input Validation & Output Encoding**
 - Input Validation and Output Encoding are essential for preventing common web vulnerabilities like XSS, SQL Injection, and command injection
- **Input Validation:**
 - Ensure that user-supplied data is correct, safe, and expected before using it.

Security best practices for web development

- Validate on both client and server sides
 - Client-side: For user convenience.
 - Server-side: For security (never trust client input alone).
- Use Whitelisting (Allowlisting)
 - Only allow known good values.
 - Example: Accept a-z, 0-9, specific lengths, patterns (e.g., email format).
- Use Built-in Validators
 - Frameworks like Flask-WTF, Django Forms, or Express-Validator help validate types and ranges.

Security best practices for web development

- Type & Range Checking
 - Ensure numbers are within expected bounds, strings have proper length, and dates are valid.
- Avoid Blacklisting (Denylisting)
 - It's easy to bypass and hard to maintain.
- Sanitize Input (if needed)
 - Especially for rich text editors, file uploads, etc.
- Output Encoding
 - Neutralize special characters before rendering them to prevent injection attacks (e.g., XSS).

Security best practices for web development

- Escape all dynamic content
 - Especially when displaying user input in HTML, JavaScript, CSS, or URLs.
- Use Context-Aware Encoding
 - HTML context: Escape `<`, `>`, `&`, `"`
 - JavaScript context: Escape quotes and backslashes
 - Attribute context: Escape `"` and `'`
 - URL context: Use `encodeURIComponent()` or equivalent
- Use Templating Engines that auto-escape
 - Jinja2 (Flask), Thymeleaf (Java), Razor (ASP.NET) by default escape HTML content.
- Never disable auto-escaping unless absolutely needed and safe.

Secure Authentication and Authorization

- Authentication and authorization are core components of web application security:
 - Authentication verifies who the user is.
 - Authorization verifies what the authenticated user is allowed to do.

Secure Authentication and Authorization

- Multi-Factor Authentication (MFA)
 - MFA requires users to provide two or more verification factors to gain access (e.g., password + OTP or biometric).
- Best Practices:
 - Use TOTP apps (e.g., Google Authenticator) or hardware tokens.
 - Avoid relying solely on SMS (can be intercepted).
 - Allow backup methods like email codes or recovery codes.
 - Enforce MFA for admin-level access or sensitive operations.

Secure Authentication and Authorization

- **Role-Based Access Control (RBAC)**

- RBAC restricts system access based on the roles assigned to users (e.g., admin, user, editor).

- **Best Practices:**

- Define clear roles and associated permissions.
- Use server-side checks for authorization.
- Prevent privilege escalation (e.g., a user becoming admin via URL manipulation).
- Combine RBAC with attribute-based access if needed (ABAC).

- **Example:**

```
if current_user.role != 'admin':  
    abort(403)
```

Secure Authentication and Authorization

- **Secure Session Handling**

- Ensure sessions (e.g., login sessions) can't be hijacked or stolen.

- **Best Practices:**

- Use **HTTPS only** to prevent session token interception.
 - Mark session cookies with:
 - Secure (HTTPS only)
 - HttpOnly (not accessible via JavaScript)
 - SameSite=Strict or Lax (CSRF protection)
 - Store session tokens in secure server-side stores (not localStorage).
 - Expire sessions after inactivity or logout.
 - Regenerate session ID after login (session fixation protection).

Secure Authentication and Authorization

- **Session Fixation and Hijacking Prevention**
 - **Session Fixation:** Attacker sets a known session ID before login.
 - Fix: Regenerate session ID after login.
- **Session Hijacking:** Attacker steals a valid session token.
 - Fixes:
 - Use HTTPS.
 - Secure and HttpOnly cookies.
 - Monitor for abnormal behavior (IP change, user-agent mismatch).

Secure Authentication and Authorization

- **Error Handling and Logging**

- Prevent information leakage while maintaining traceability.

- **Best Practices:**

- Show generic error messages to users (e.g., "Something went wrong").
 - Log detailed errors **securely on the server side** (never show stack traces to users).
 - Avoid leaking sensitive details (file paths, DB info, API keys).
 - Sanitize log inputs to avoid log injection attacks.
 - Use centralized logging and monitoring (e.g., ELK stack, Sentry).

Secure Authentication and Authorization

- **Secure File Uploads**

- Malicious files (e.g., scripts) uploaded and executed on the server.

- **Best Practices:**

- Limit allowed file types (e.g., .jpg, .pdf).
- Use MIME type and file header validation—not just file extension.
- Rename uploaded files to avoid original names (e.g., UUIDs).
- Store uploads **outside the web root** to prevent direct access.
- Set appropriate permissions (e.g., read-only).
- Scan files with antivirus software.

Secure Authentication and Authorization

- **SSL/TLS**
 - **SSL (Secure Sockets Layer)** and **TLS (Transport Layer Security)** are cryptographic protocols that **secure communication over a network**.
 - **TLS** is the modern, secure replacement for SSL.
 - They provide:
 - **Encryption** – Prevents data from being read during transmission.
 - **Integrity** – Ensures data is not altered.
 - **Authentication** – Confirms the identity of the server (and optionally the client)

Secure Authentication and Authorization

- **HTTPS**

- **HTTPS (Hypertext Transfer Protocol Secure) = HTTP + SSL/TLS**
- It ensures that **data sent between a browser and web server is encrypted and secure.**
- HTTPS uses **port 443** (HTTP uses port 80).

Firewalls

- A **firewall** is a security device or software that **monitors and controls incoming and outgoing network traffic** based on predetermined security rules.
- Acts as a **barrier** between a trusted internal network and untrusted external networks (like the internet).
- **There are different types of firewall:**
 - **Network Firewalls:** Hardware or software protecting an entire network.
 - **Host-based Firewalls:** Software running on individual devices.

Firewalls

- **Next-Generation Firewalls (NGFW):** Include advanced features like intrusion prevention, deep packet inspection.
- **The purpose of using firewall:**
 - Block unauthorized access.
 - Filter traffic based on IP addresses, ports, protocols.
 - Protect against attacks like malware, DoS, unauthorized access.

Proxies

- A proxy server acts as an intermediary between a client and the internet.
- Requests from clients go to the proxy, which forwards them to the target server, then returns responses to the client.
- **There are two types of proxies:**
 - **Forward Proxy:** Used by clients to access the internet anonymously or to filter outgoing traffic.

Secure Authentication and Authorization

- **Reverse Proxy:** Sits in front of web servers to distribute load, provide caching, and protect backend servers.
- **The purpose of using proxies:**
 - Hide client IP addresses.
 - Filter or cache content.
 - Improve performance and security.
 - Enforce access control policies.

VPNs (Virtual Private Networks)

- A **VPN** creates a **secure, encrypted tunnel** between the user's device and a VPN server over the internet.
- Masks the user's IP address and encrypts all traffic to protect privacy.
- **The Purpose of using VPN:**
 - Secure communication over untrusted networks (e.g., public Wi-Fi).
 - Bypass geo-restrictions and censorship.
 - Protect sensitive data from interception.

Host Security Threats and Countermeasures

Threat	Description
Malware	Includes viruses, worms, trojans, ransomware that compromise host systems.
Unauthorized Access	Attackers gaining access to host systems without permission.
Privilege Escalation	Attackers gain higher-level permissions than intended.
Data Theft or Loss	Sensitive files or credentials being stolen, deleted, or modified.
Rootkits	Software that hides malicious activity, often with admin/root access.
Unpatched Vulnerabilities	Security holes due to outdated OS or applications.

Host Security Threats and Countermeasures

Countermeasure	Description
Antivirus & Antimalware	Detects and removes malicious software.
Host-based Firewalls	Controls incoming/outgoing traffic to/from host.
Regular Patching	Keep OS and software up-to-date.
Access Control	Use strong passwords, multi-factor authentication, and user privileges.
System Hardening	Disable unnecessary services, ports, and accounts.
Host Intrusion Detection System (HIDS)	Monitors and alerts suspicious activities.
Disk Encryption	Protects data even if physical access to device is gained.

Network Security Threats and Countermeasures

Threat	Description
Eavesdropping / Sniffing	Intercepting data during transmission (e.g., Wi-Fi sniffing).
Man-in-the-Middle (MITM)	Attacker intercepts and possibly alters communication between two parties.
Denial of Service (DoS/DDoS)	Overwhelms the network to disrupt services.
IP Spoofing	Pretending to be another IP to bypass security measures.
ARP Spoofing	Associating attacker's MAC with a legitimate IP.
DNS Poisoning	Redirecting traffic to malicious sites.
Unauthorized Access	Intrusion into the network via open ports or weak credentials.

Network Security Threats and Countermeasures

Countermeasure	Description
Network Firewalls	Filter and control traffic between trusted and untrusted networks.
Intrusion Detection/Prevention Systems (IDS/IPS)	Detects or blocks malicious traffic.
VPN (Virtual Private Network)	Encrypts traffic to protect against eavesdropping.
Network Segmentation	Isolates critical parts of the network.
TLS/SSL Encryption	Secure web and email communications.
Secure Configuration	Disable unused ports and protocols, use strong credentials.
Monitoring & Logging	Continuously inspect logs for unusual activity.

Application Security threats and countermeasures

Threat	Description
SQL Injection (SQLi)	Attacker injects malicious SQL code to manipulate the database.
Cross-Site Scripting (XSS)	Attacker injects malicious scripts into web pages viewed by others.
Cross-Site Request Forgery (CSRF)	Tricks a user into executing unwanted actions on a trusted site.
Broken Authentication	Attackers exploit flaws in login/session management to impersonate users.
Sensitive Data Exposure	Insecure storage or transmission of personal or confidential data.
Security Misconfiguration	Poorly configured servers, frameworks, or software (e.g., default credentials).
Insecure Deserialization	Executing untrusted or tampered data that can lead to remote code execution.
Using Components with Known Vulnerabilities	Outdated libraries or frameworks with known security flaws.
Insufficient Logging & Monitoring	Attacks go undetected due to lack of logs or real-time monitoring.

Application Security threats and countermeasures

Countermeasure	Description
Input Validation & Sanitization	Validate all user inputs; use allowlists; sanitize output to prevent injection.
Use Parameterized Queries / ORM	Prevents SQL injection by separating data from SQL logic.
Output Encoding	Encode data before displaying it in a browser to prevent XSS.
Implement CSRF Tokens	Random tokens in forms to prevent unauthorized requests.
Use Strong Authentication	Enforce MFA, secure session tokens, and login throttling.
HTTPS Everywhere	Encrypt data in transit using TLS/SSL.
Secure Cookies	Use <code>HttpOnly</code> , <code>Secure</code> , and <code>SameSite</code> flags on cookies.
Keep Components Updated	Regularly patch and update dependencies and libraries.
Use Security Headers	Enforce browser security with headers like CSP, HSTS, X-Frame-Options, etc.
Enable Logging & Alerting	Monitor access logs and set up alerts for unusual behavior.
Perform Code Reviews & Penetration Testing	Regularly audit source code and test for vulnerabilities.
Use Web Application Firewall (WAF)	Filters and monitors HTTP traffic to prevent common attacks.

Secure Development Practices (SDLC)

Stage	Security Action
Requirements	Define security requirements (e.g., OWASP Top 10 threats).
Design	Use threat modeling and secure design principles.
Implementation	Follow secure coding guidelines and conduct peer reviews.
Testing	Perform security testing: SAST, DAST, fuzzing, etc.
Deployment	Harden deployment environments and limit access.
Maintenance	Monitor, log, patch, and respond to incidents promptly.

Design guidelines for secure web applications

- Design Guidelines for Secure Web Applications, focusing on security from the ground up.
- These principles help ensure your web apps are robust against common attacks like XSS, SQLi, CSRF, and more.
- Some of the guidelines for secure web applications
 - Input Validation and Output Encoding
 - Authentication and Session Management
 - Access Control

Design guidelines for secure web applications

- Secure Communication
- Error and Exception Handling
- Secure Configuration
- Prevent Common Vulnerabilities
- Secure File Uploads
- Secure Dependencies
- Logging and Monitoring
- Security Testing

Next Chapter